Chapter 1
PRELIMINARY DESIGN

1.1 Introduction

S K BIKE POINT is a modern, enterprise-grade two-wheeler workshop and customer management web platform engineered to modernize automobile garages and service centers. Traditional neighborhood two-wheeler workshops operate almost entirely on manual paper job cards, handwritten cost estimates, physical inventory ledgers, and untracked service records. This leads to customer anxiety over repair turnaround times, untracked spare parts leakage, tax inaccuracies, and lost customer retention due to lack of periodic maintenance reminders.

The system provides a cloud-native 3-tier architecture with Zero-Login Live Vehicle Tracking (allowing bike owners to check real-time 5-stage repair progress using only their vehicle number plate), digitized workshop job-card workflows, automated inventory threshold decrementing, and GST 18% compliance billing. The application is built with a responsive React 18 and Tailwind CSS frontend, a robust Java 21 and Spring Boot 3 REST API backend, and a cloud-managed PostgreSQL 16 database hosted on Supabase. Security is enforced through stateless JWT authentication, BCrypt password hashing, and role-based access control.

1.2 Objectives

- **Digital Job Card Management:** Digitize customer intake, vehicle diagnostic recording, technician assignments, and repair stage progression.
- **Zero-Login Live Tracking:** Empower bike owners to view live 5-stage repair progress, mechanic diagnostics, and parts estimates by entering their number plate without mandatory registration.
- **Smart Spares Inventory:** Automate stock tracking for spare parts and lubricants with low-stock replenishment triggers and rack location indexing.
- **Automated GST 18% Invoicing:** Auto-calculate parts cost, labor charges, and 18% GST (9% CGST + 9% SGST) with instant tax invoice PDF preview and printing.
- **Executive Workshop Dashboard:** Provide garage owners with real-time analytics covering daily revenue, active floor jobs, low-stock alerts, and technician utilization.
- **Customer Digital Garage:** Allow registered customers to track multiple owned bikes, service histories, preventive maintenance intervals, and past invoices.
- **Online Service Appointment Booking:** Simple service scheduling form allowing riders to choose dates, slots, and service packages online.
- **Role-Based Security:** Strict authorization separating Workshop Administrator (ROLE_ADMIN) and Customer (ROLE_CUSTOMER) privileges.

1.3 Scope and Limitations of the Project

The S K BIKE POINT platform is designed to establish an end-to-end digital lifecycle for two-wheeler service workshops, bridging the communication gap between motorcycle riders, floor mechanics, and garage proprietors. Defining a precise operational boundary ensures focused implementation, robust stability, and predictable production deployments.

A. Functional Scope (Included Modules):

- **Zero-Login Live Vehicle Tracker:** Enables any motorcycle or scooter owner to track live 5-stage repair progression, assigned technician, and parts cost estimates using only their vehicle registration number, completely removing login friction.
- **Digital Job Card Lifecycle:** Digitizes intake inspection, mechanic assignment, diagnostic note capture, and stage transitions (Intake, Spares Requisition, Assembly & Road Testing, Washing & Invoicing, Delivered).
- **Smart Inventory & Spares Ledger:** Maintains real-time stock balances across consumable categories, automatically decrements stock upon job allocation, and generates visual low-stock alert badges.
- **Automated GST 18% Tax Billing:** Generates compliant tax invoices calculating 9% CGST + 9% SGST on billable labor and spare parts, with instant PDF download and payment settlement tracking.
- **Executive Workshop Analytics:** Provides garage proprietors with live KPIs including active floor jobs, technician utilization, daily gross revenue, and pending invoices.

B. Project Limitations (Boundary Constraints):

- **Single-Location Garage Operations:** The initial release is architected for single-branch workshops (such as S K BIKE POINT) and does not currently manage cross-warehouse inter-branch transfers.
- **Online Payment Processing:** Payment settlement currently registers transaction confirmation modes (Cash, UPI QR, Card) and prepares digital GST receipts, but does not integrate third-party banking escrow gateways.
- **SMS Gateway Simulation:** Real-time repair notifications stream via zero-login web sockets and responsive URLs; cellular SMS dispatch is simulated via on-screen instant alerts.

1.4 Hardware and Software Requirements Specification

The system is designed with a lightweight, cloud-native footprint, allowing workshop counter staff and floor mechanics to operate the application on budget-friendly desktop PCs, tablets, or mobile phones, while leveraging scalable cloud infrastructure for backend microservices and relational persistence.

Table 1.4.1: Hardware Requirements Specification

Component	Minimum Requirement	Recommended Specification
Processor (CPU)	Dual Core Intel Core i3 / AMD Ryzen 3 (2.0 GHz)	Quad Core Intel Core i5 / AMD Ryzen 5 (3.0 GHz+)
Memory (RAM)	4 GB DDR4 RAM for counter client terminals	8 GB – 16 GB DDR4/DDR5 for development and local server
Hard Disk / Storage	256 GB SATA SSD (at least 20 GB free space)	512 GB NVMe M.2 SSD for rapid asset caching and builds
Display Resolution	1366 x 768 standard desktop / 768 x 1024 tablet	1920 x 1080 Full HD IPS display for executive analytics
Network Connectivity	Broadband internet (minimum 10 Mbps)	Stable high-speed broadband (50 Mbps+ with Wi-Fi 6)
Client Peripherals	Standard keyboard, mouse, barcode scanner	Laser printer or 80mm thermal receipt slip printer

Table 1.4.2: Software Requirements Specification

Layer / Environment	Technology / Software Specification	Version / Purpose
Operating System	Microsoft Windows 11 / 10 Pro or Ubuntu Linux 22.04 LTS	Host OS for local development and cloud container execution
Backend Java SDK	Eclipse Adoptium OpenJDK 21 LTS (Java 21 HotSpot)	Provides virtual threads, record patterns, and modern performance
Backend Framework	Spring Boot Enterprise Framework with Spring Web MVC	Version 3.3.3 for building robust stateless REST API endpoints
Security Middleware	Spring Security 6 with JJWT 0.12.6 (HMAC-SHA256)	Enforces stateless JWT authentication and role-based guards
Database Tier	PostgreSQL 16 Relational Database Engine on Supabase	Cloud-hosted persistence on AWS ap-south-1 Mumbai over TLS
Connection Pooler	HikariCP JDBC Connection Pooler	Manages high-performance SQL pooling with 10 max connections

Frontend Engine	React 18 Single Page Application with TypeScript 5.5	Provides reactive UI state management and type safety
Build Tool & Bundler	Vite 5 Bundler and Node.js 20 LTS runtime	Delivers lightning-fast hot module replacement and tree shaking

1.5 Stakeholders

Stakeholder	Role and Actions
Technical Team	Develop, maintain, secure, and deploy the React frontend, Spring Boot API, and Supabase cloud database. Ensure system uptime and TLS encryption.
Workshop Mechanics / Floor Staff	Inspect two-wheelers, log diagnostic checklists, requisition spares from the store, execute repairs, and update live 5-stage workflow status.
End Users (Bike Owners / Customers)	Track real-time repair progress via number plate, view diagnostic notes, book service appointments, and download GST tax invoices.
Workshop Owner / Client (Sanjay Yadav)	Oversee daily shop operations, track gross revenue and technician throughput, manage inventory purchasing, and approve gate passes.
System Architect & Developer (Sumeet Yadav)	Design domain models, implement REST endpoints, configure JPA repositories, integrate frontend routing, and prepare technical documentation.

1.6 System in use (Comparative Study)

Feature	Traditional Garages	Existing Workshop Softwares	S K BIKE POINT (Proposed)
Job Card Management	Manual carbon-copy paper job cards	Desktop-bound legacy billing software	Cloud-native digital job cards with real-time 5-stage progress tracking
Vehicle Repair Tracking	Uncertain phone inquiries and counter visits	Manual SMS notification or no client portal	Instant Zero-Login live tracking by bike number plate or job number
Spares Inventory	Manual register books; untracked stock leakage	Basic standalone offline stock ledger	Automated job-linked stock deduction with low-stock replenishment alert
Billing & Invoicing	Handwritten paper slips with estimated tax	Generic accounting invoices without bike details	Automated GST 18% (9% CGST + 9% SGST) invoice with itemized parts & labor

Customer History	Lost physical records; no maintenance log	Scattered customer contact spreadsheets	Centralized digital garage with complete historical service records
Authentication & Security	No security controls; open physical access	Basic password login without role enforcement	Stateless JWT token authentication with BCrypt hashing & role authorization

1.6 System in use (Comparative Study - Continued)

Feature	**Traditional Garages**	**Existing Workshop Softwares**	**S K BIKE POINT (Proposed)**
Database	Physical paper files and notebooks	Local single-PC Microsoft Access / SQLite	Cloud PostgreSQL 16 on Supabase with encrypted SSL connection pooling
Low-Stock Alerts	Discovered only when part runs out mid-repair	End-of-month manual stock take reports	Real-time alert badges when stock reaches minimum reorder threshold
Appointment Booking	Verbal promises or physical queue on arrival	Not supported / external call desk	Interactive online appointment scheduling with time-slot selection
Accessibility	Accessible only physically inside workshop	Restricted to desktop LAN connection	Fully cloud-accessible on mobile, tablet, and desktop browsers
Security Controls	None; prone to loss and ledger tampering	Unencrypted local storage	HTTPS, CORS whitelist, JWT expiration, and encrypted credentials
Overall Benefit	Slow, unorganized, and prone to revenue leak	Rigid, desktop-tied, and poor user UX	Centralized, transparent, and high-efficiency workshop operations

1.7 Technology Stack

Layer	**Technology**
Frontend Framework	React 18 Single Page Application with TypeScript 5.5
Build Tool & Bundler	Vite 5 (Lightning-fast HMR and optimized production bundling)

Styling & Design System	Tailwind CSS 3.4 with custom workshop dark/light tokens
Client Routing & Icons	React Router DOM 6.26 & Lucide React 0.438
Data Visualization	Recharts 2.12 (Interactive workshop revenue & volume trend charts)
Backend Application Server	Java 21 LTS with Spring Boot 3.3.3 Enterprise REST Framework
Security & Auth	Spring Security 6 with JJWT 0.12.6 (HMAC-SHA256) & BCrypt
Data Access / ORM	Spring Data JPA & Hibernate 6 with HikariCP Connection Pooling
Cloud Database	PostgreSQL 16 hosted on Supabase (AWS ap-south-1 Mumbai)

Chapter 2
SYSTEM ANALYSIS

2.1 Work Breakdown Structure (WBS)

The development of the S K BIKE POINT workshop platform is organized into six structured engineering phases:

1. Requirements and System Planning: Field research in two-wheeler workshops; stakeholder interviews with garage owners and mechanics; analysis of common job card bottlenecks; preparation of the Software Requirements Specification (SRS); determination of functional and non-functional requirements.

2. System Architecture & Relational Database Design: Formulation of the 3-tier system architecture; ER modeling for 7 core entities (Users, Vehicles, Job Cards, Inventory, Invoices, Appointments, Job Card Parts); normalization to 3NF; schema indexing and foreign key constraints.

3. Backend REST API Development: Implementation of Spring Boot 3.3.3 REST controllers; Spring Security 6 with stateless JWT Bearer token authentication; service logic for job tracking, inventory decrements, and GST calculation; database seeding with mock data.

4. Frontend Web Application Development: Creation of React 18 single page application with Tailwind CSS; development of Zero-Login Live Vehicle Tracker, Admin Command Center, Service Jobs Kanban queue, Spares Inventory ledger, and Customer Garage portal.

5. Cloud Deployment & Container Orchestration: Docker containerization with multi-stage JDK 21 Alpine and Nginx builds; deployment of backend on Render; deployment of frontend on Vercel CDN; configuration of Supabase PostgreSQL cloud database.

6. Testing, Quality Assurance & Documentation: Unit and integration testing; master test case execution across all features; validation of security access controls; generation of UML diagrams, user guides, and comprehensive academic project documentation.

2.2 Functional and Non-Functional Requirements (SRS)

A rigorous Software Requirements Specification establishes the precise capabilities and behavioral guarantees demanded of the S K BIKE POINT workshop system. Requirements are categorized into Functional Requirements (system features) and Non-Functional Requirements (quality attributes).

Table 2.2.1: Functional Requirements Specification

Req ID	Requirement Description	Category	Target User / Entity
FR-01	Zero-Login Vehicle Tracking: Search active job by license plate or job code without login.	Core Feature	Guest Rider / Customer
FR-02	Digital Job Card Lifecycle: Create, assign mechanic, record diagnostics, advance 5 stages.	Workshop Flow	Workshop Admin / Advisor
FR-03	Real-Time Spares Decrement: Auto-deduct inventory quantity upon part allocation to job.	Inventory	Storekeeper / Mechanic
FR-04	Consumable Low-Stock Alerts: Trigger visual alert badges when stock falls below threshold.	Inventory	Workshop Proprietor
FR-05	Automated GST 18% Billing: Compute 9% CGST + 9% SGST on parts and labor charges.	Invoicing	Billing Desk / Customer
FR-06	Customer Digital Garage: Maintain indelible maintenance history and invoices for owned bikes.	Self-Service	Registered Bike Owner
FR-07	Online Appointment Scheduling: Select vehicle model, service bay slot, and preferred date.	Booking	Prospective Rider
FR-08	Role-Based Access Control: Enforce strict authorization boundaries (ADMIN and CUSTOMER).	Security	Spring Security 6 / JWT

Table 2.2.2: Non-Functional Requirements Specification

NFR ID	Quality Attribute	Specification & Metric Guarantee
NFR-01	Performance & Latency	API endpoints must deliver 95th percentile response times below 200 ms under standard workshop load.
NFR-02	Security & Data Protection	Passwords must be hashed using BCrypt (cost factor 10); JWT tokens signed via HMAC-SHA256 with 24-hr TTL.
NFR-03	Availability & Uptime	Cloud deployment on Render and Supabase must maintain 99.9% uptime with automated health

		probes.
NFR-04	Data Integrity & ACID	All inventory decrements and invoice generations must execute within atomic database transactions.
NFR-05	Responsive Usability	UI must seamlessly adapt across mobile viewports (375px), workshop tablets (768px), and counter monitors (1920px).
NFR-06	Maintainability & Modularity	Codebase must adhere to 3-tier layering with strict decoupling between UI controllers, services, and repositories.

2.3 Gantt Chart and Schedule

The project development lifecycle spanned 10 weeks, following an agile milestone schedule from requirements gathering to cloud deployment and final verification.

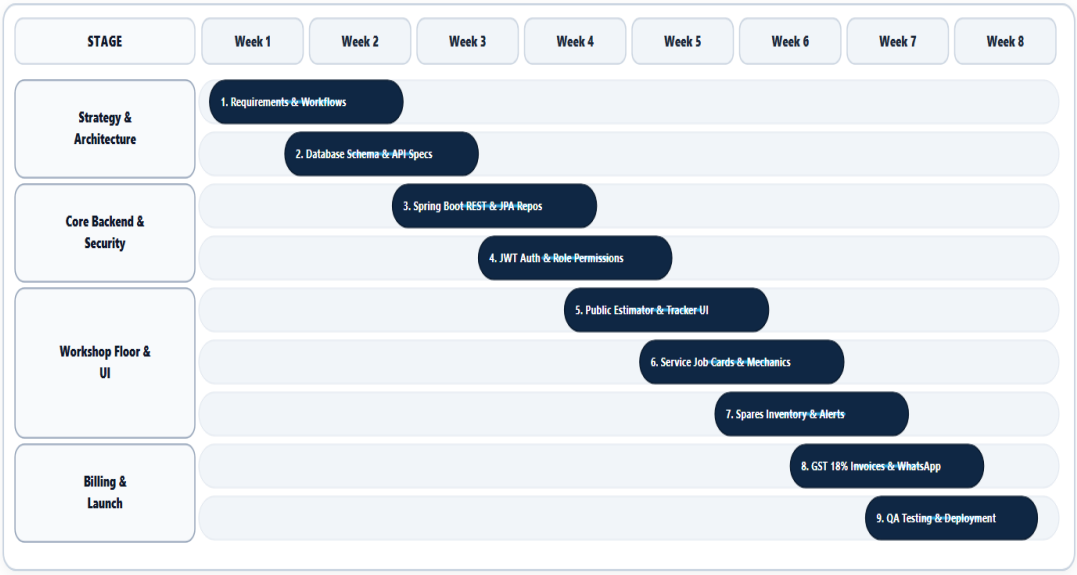


Figure 2.1 – Project Development Gantt Chart & Schedule

Phase / Milestone	Time Period	Key Deliverables
Phase 1: Requirements	Weeks 1 – 2 (Aug 01 – Aug 14)	Stakeholder interviews, SRS document, technology stack selection.
Phase 2: System Design	Weeks 3 – 5 (Aug 15 – Sep 06)	ER diagrams, UML class and sequence models, UI wireframes.
Phase 3: Backend APIs	Weeks 6 – 8 (Sep 07 – Sep 28)	Spring Boot REST endpoints, JPA entities, JWT security filters.
Phase 4: Frontend App	Weeks 9 – 10 (Sep 29 – Oct 05)	React SPA, Admin dashboard, Public tracker, responsive styling.
Phase 5: Cloud & QA	Week 10 (Oct 05 – Oct 08)	Supabase DB, Render & Vercel deployment, integration testing.

2.4 Domain Class Diagram

The analysis model identifies seven core domain classes: User, Vehicle, JobCard, JobCardPart, InventoryItem, Invoice, and Appointment. The User entity models system actors (Admin and Customer). A User owns multiple Vehicles and can schedule Appointments. A Vehicle undergoes maintenance through JobCards. Each JobCard aggregates multiple JobCardParts, which decrement InventoryItems, and upon service completion generates a GST compliant Invoice.

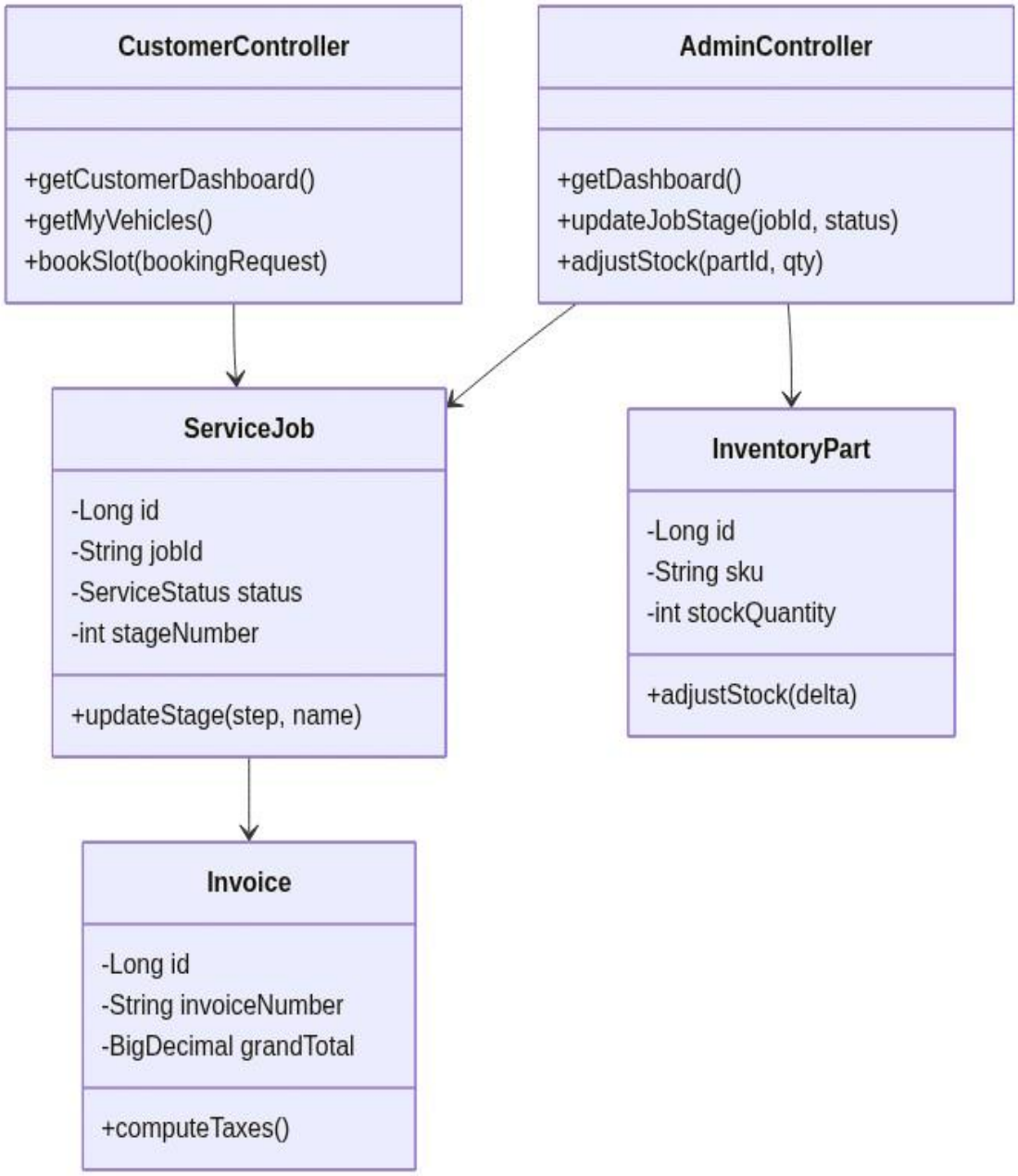


Figure 2.2 – Domain Class Diagram

Chapter 3
SYSTEM DESIGN

3.1 Use Case Diagram and Specification

Figure 3.1 illustrates the system boundary and interactions between the four key actors: Guest Rider, Registered Customer, Workshop Mechanic, and Workshop Administrator. Unregistered riders can track repair progress zero-login and view estimates. Customers manage owned bikes and past records. Mechanics update repair stages and requisition parts. Administrators have full command over job cards, inventory alerts, and GST billing.

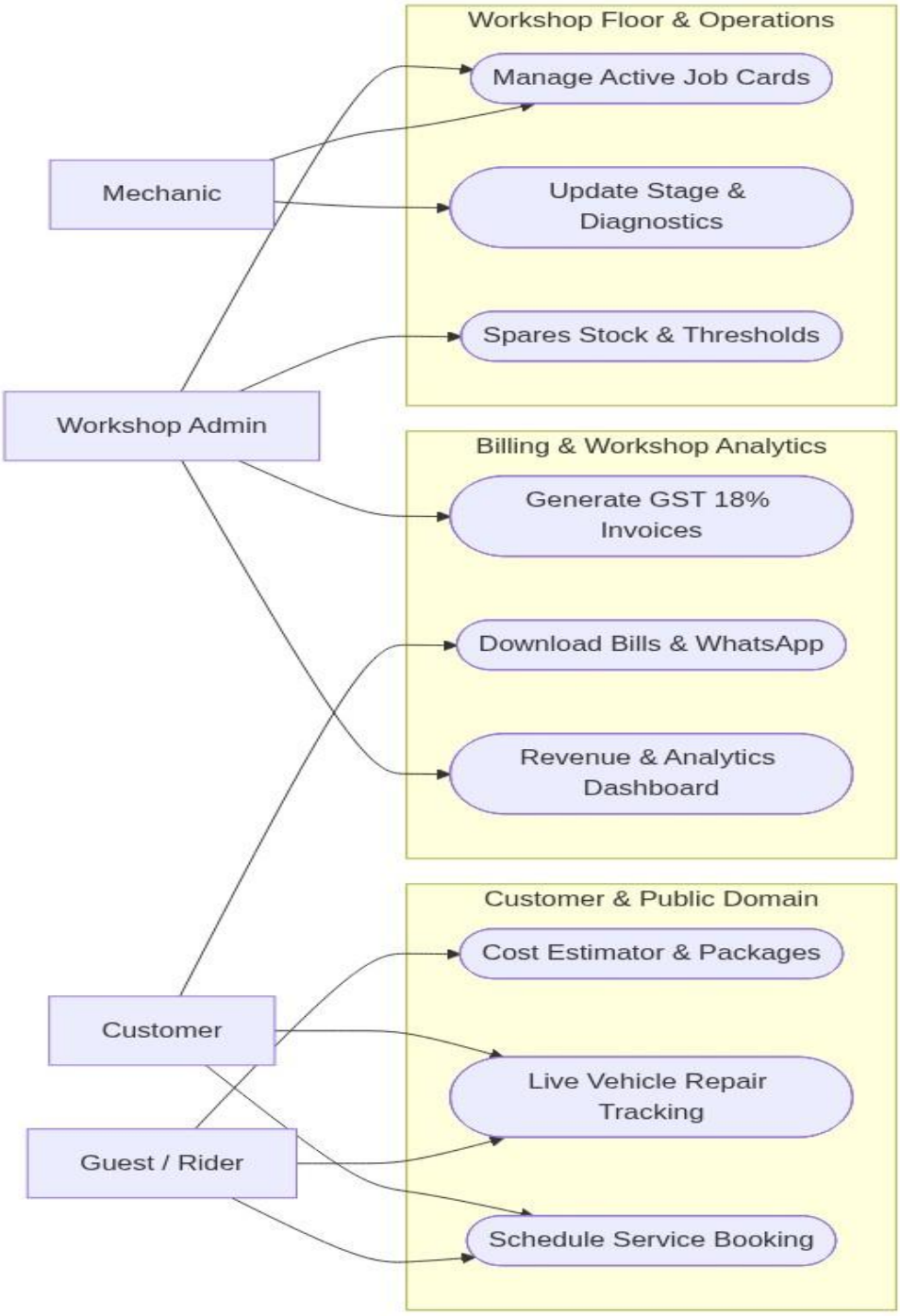


Figure 3.1 – Use Case diagram

Table 3.1 – Actor and Use Case Dependency Matrix

ID	Use Case	Primary Actor	Secondary Actors	Included / Extended Behaviour
UC-01	Zero-Login Vehicle Tracking	Guest Rider / Customer	PostgreSQL DB	Includes querying by number plate and rendering 5-stage progress stepper.
UC-02	Book Service Appointment	Guest Rider / Customer	PostgreSQL DB	Includes slot selection, package selection, and booking confirmation.
UC-03	Customer Digital Garage	Registered Customer	PostgreSQL DB	Includes vehicle registration, odometer mileage tracking, and invoice downloads.
UC-04	Admin / User Authentication	Admin / Customer	Spring Security, JWT	Includes email/password validation, BCrypt match, and JWT token issuance.
UC-05	Create Digital Job Card	Workshop Admin	Mechanic, Inventory	Includes mechanic assignment, initial diagnostic logging, and part linking.
UC-06	Requisition Spare Parts	Mechanic / Admin	Inventory Module	Includes stock validation and automated inventory decrement in database.
UC-07	Update 5-Stage Repair Progress	Mechanic / Admin	Live Tracker API	Includes advancing stage from Intake to Washing and Ready for Delivery.
UC-08	Inventory & Stock Alerts	Workshop Admin	PostgreSQL DB	Includes adding stock, threshold warnings, and rack bin management.
UC-09	Generate 18% GST Invoice	Workshop Admin	Tax Engine, DB	Includes calculating 9% CGST + 9% SGST, labor total, and PDF generation.

UC-10	View Executive Dashboard	Workshop Admin	Analytics Engine	Includes revenue charts, active jobs count, and critical stock warnings.

3.2 System Flowchart (Vehicle Intake & Job Card Initiation)

Figure 3.2.1 illustrates the customer intake, vehicle check-in, owner verification, and digital job card initiation process. Technicians record customer complaints, check maintenance records, and establish diagnostic checklists.

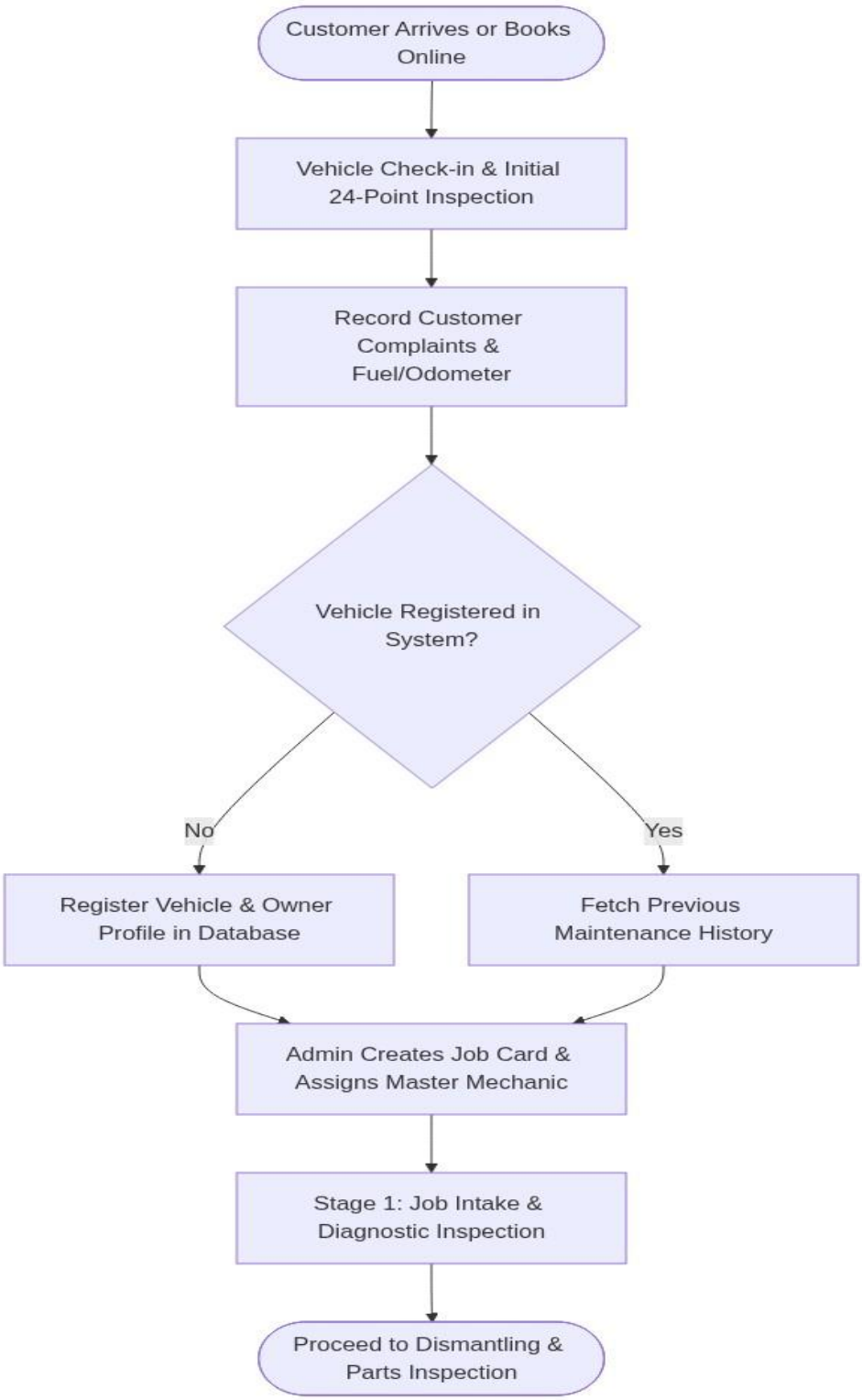


Figure 3.2.1 – System flowchart (Vehicle Intake & Job Card Initiation)

3.2 System Flowchart (Mechanical Dismantling & Spares Requisition)

Figure 3.2.2 details the physical dismantling, component wear diagnostics, and spare parts inventory allocation workflow. The system verifies real-time stock balances and decrements inventory automatically.

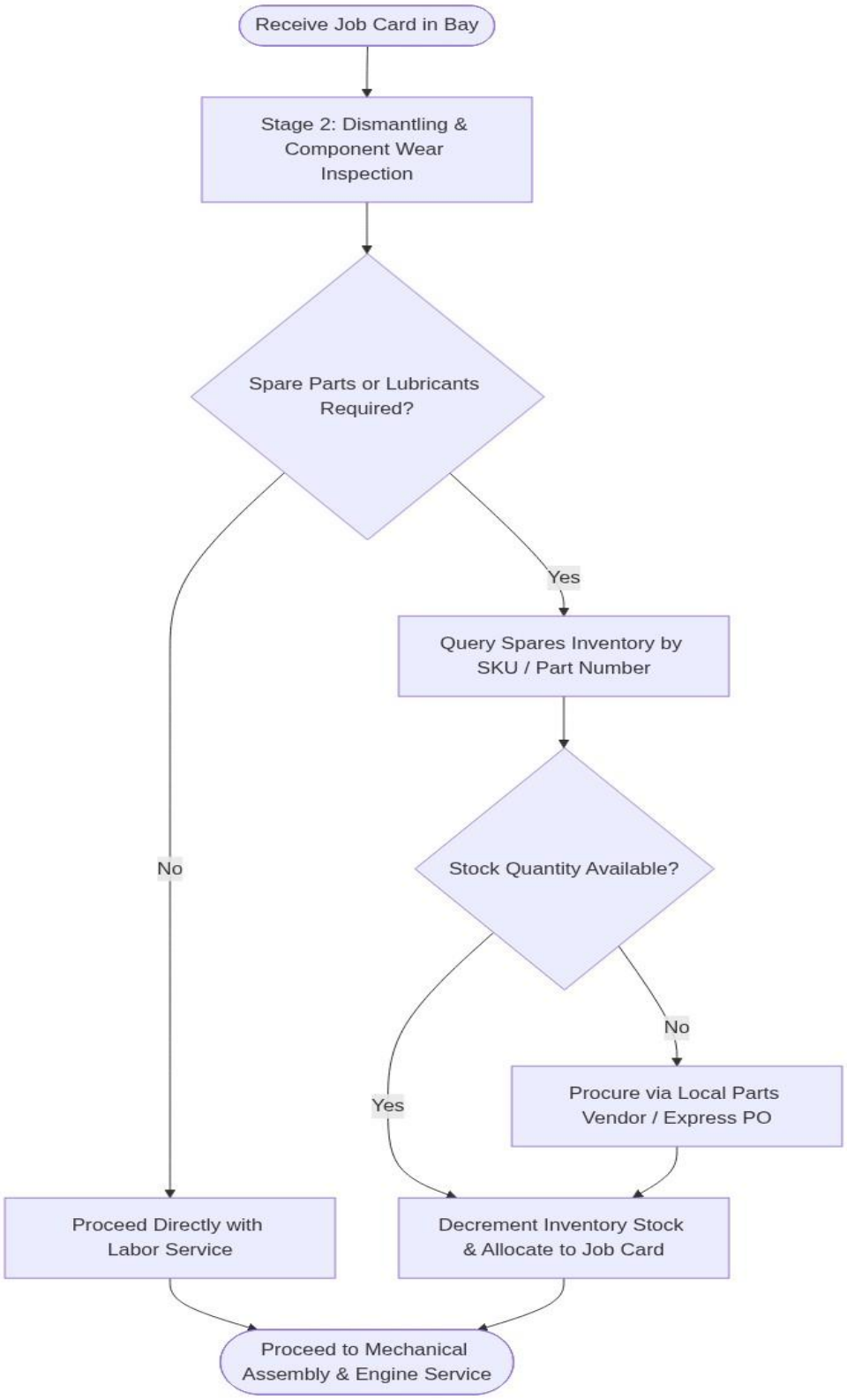


Figure 3.2.2 – System flowchart (Dismantling & Spares Requisition)

3.2 System Flowchart (Mechanical Assembly & Quality Road Testing)

Figure 3.2.3 shows the mechanical reassembly and 5-km quality control road test state machine. Master technicians inspect engine sound, braking responsiveness, and suspension tolerances before certifying the motorcycle for customer delivery.

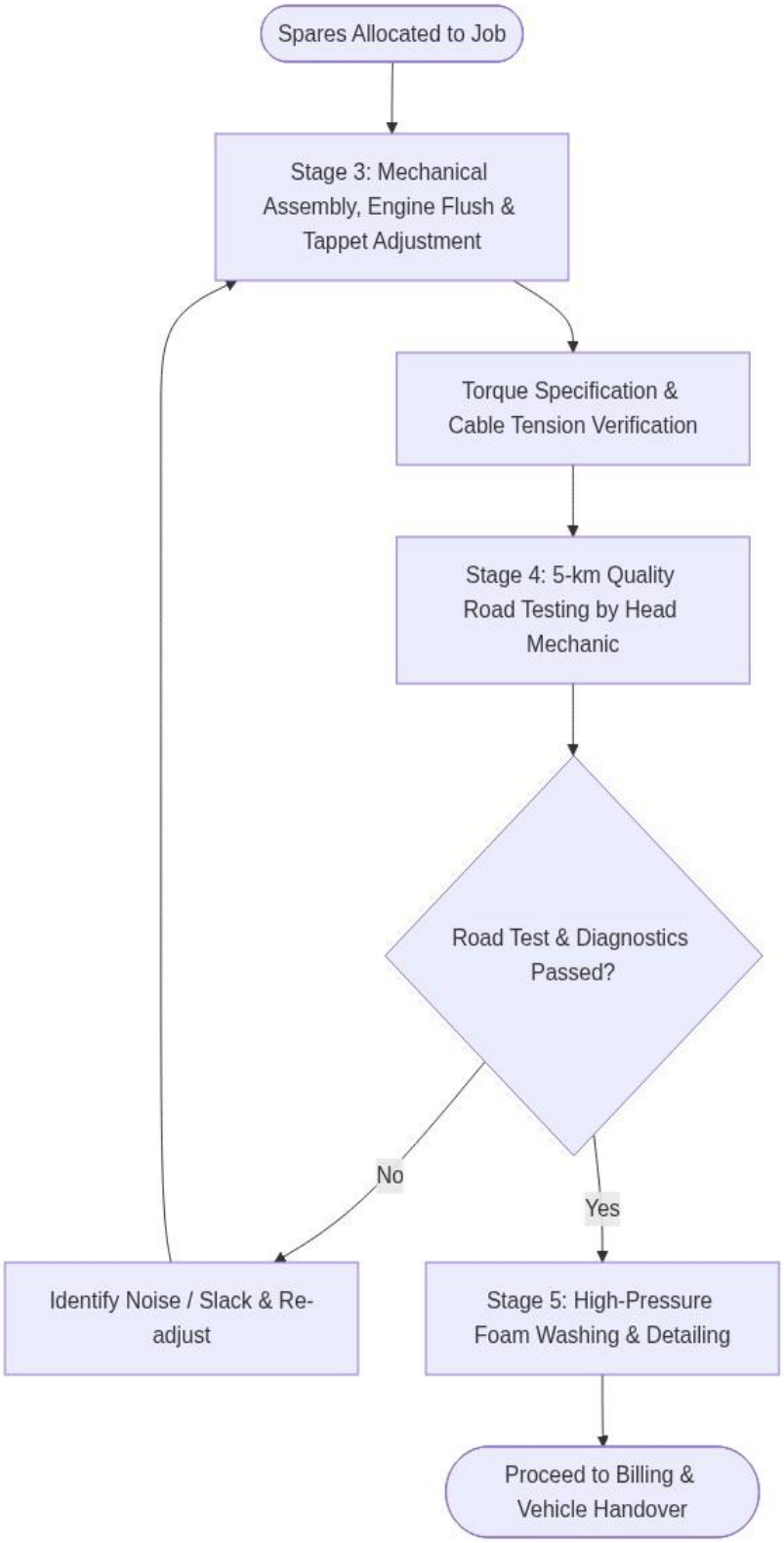


Figure 3.2.3 – System flowchart (Mechanical Assembly & Quality Road Testing)

3.2 System Flowchart (GST 18% Invoicing & Gate Pass Handover)

Figure 3.2.4 illustrates the concluding phase: high-pressure foam washing, automated GST 18% tax invoicing (9% CGST + 9% SGST), payment settlement, and physical gate pass handover to the vehicle owner.

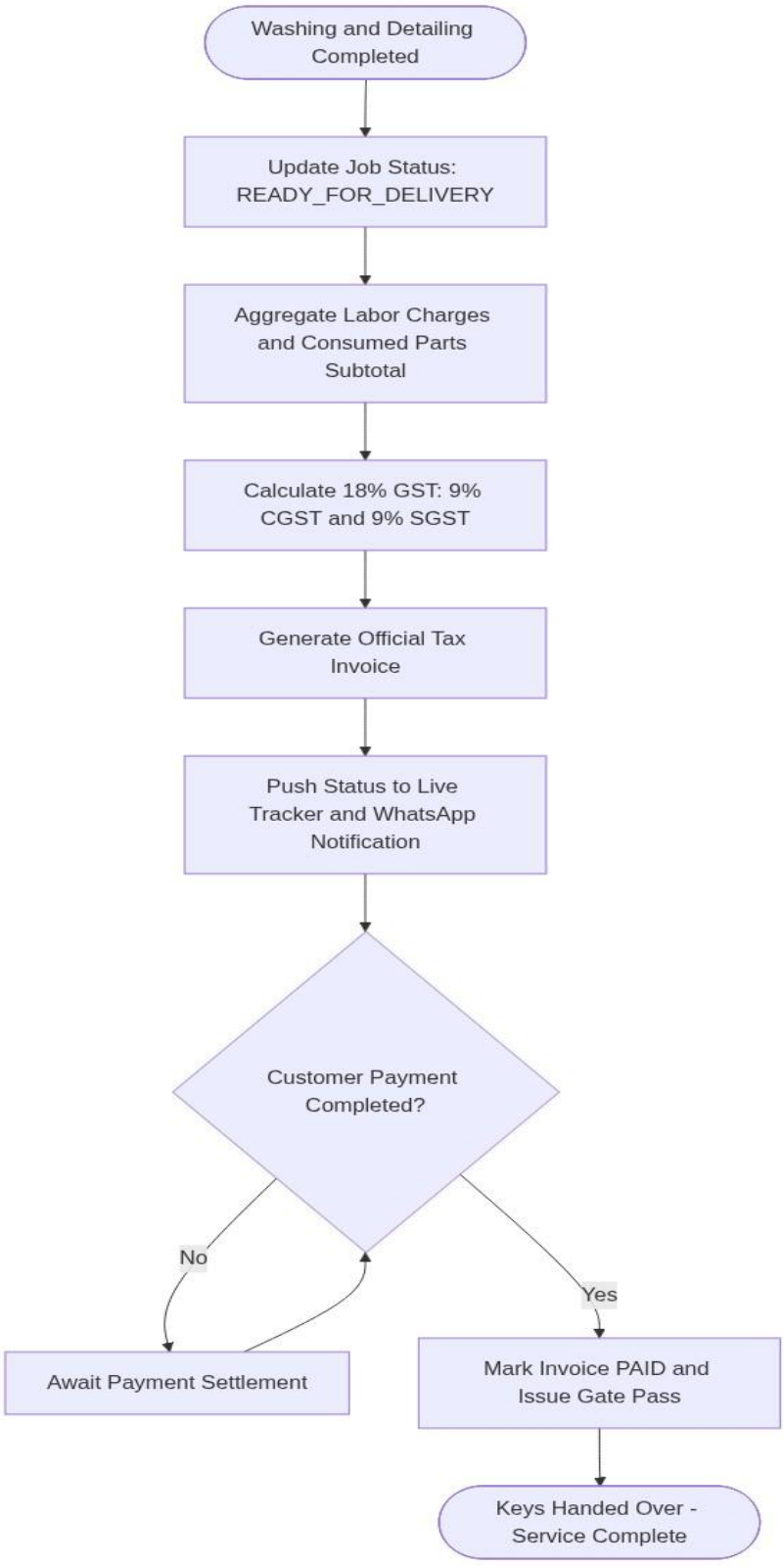


Figure 3.2.4 – System flowchart (GST 18% Invoicing & Gate Pass Handover)

3.3 Sequence Diagram (Zero-Login Live Vehicle Tracking)

Figure 3.3.1 illustrates the asynchronous sequence of public tracking calls. The unauthenticated rider submits their vehicle license plate MH-12-AB-1234. The frontend routes the request to JobCardController, which queries the database, formats the active repair stage, and returns diagnostic notes in real time.

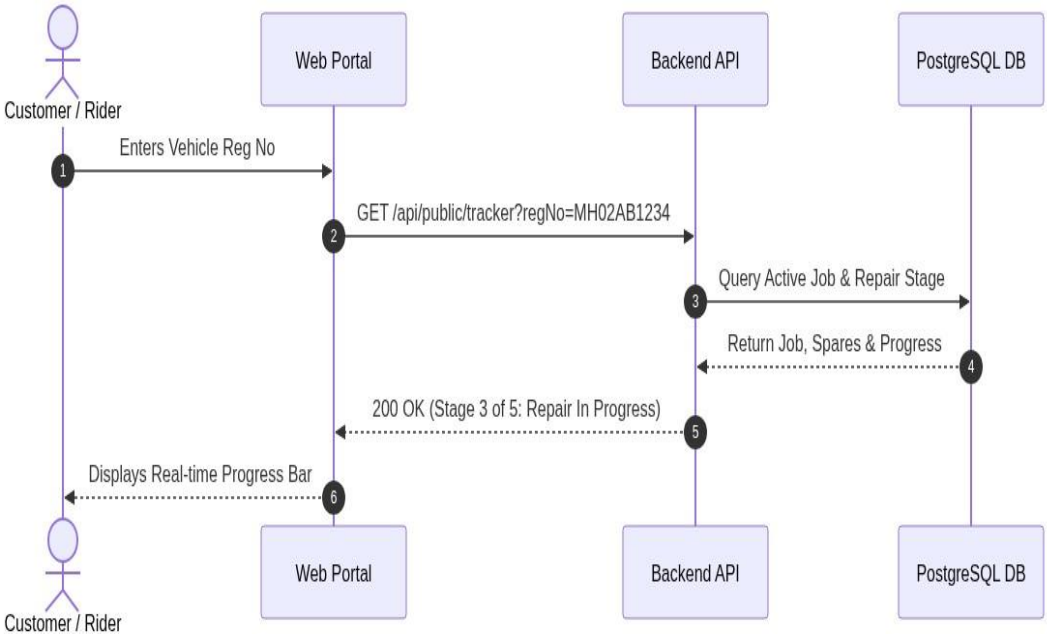


Figure 3.3.1 – Sequence Diagram (Zero-Login Live Vehicle Tracking)

3.3 Sequence Diagram (Job Card Creation & Spares Allocation)

Figure 3.3.2 traces the transactional workflow between Workshop Admin, Spring Boot REST Controllers, JobCardService, InventoryService, and the Supabase PostgreSQL database. Stock quantities are decremented atomically to prevent inventory discrepancies.

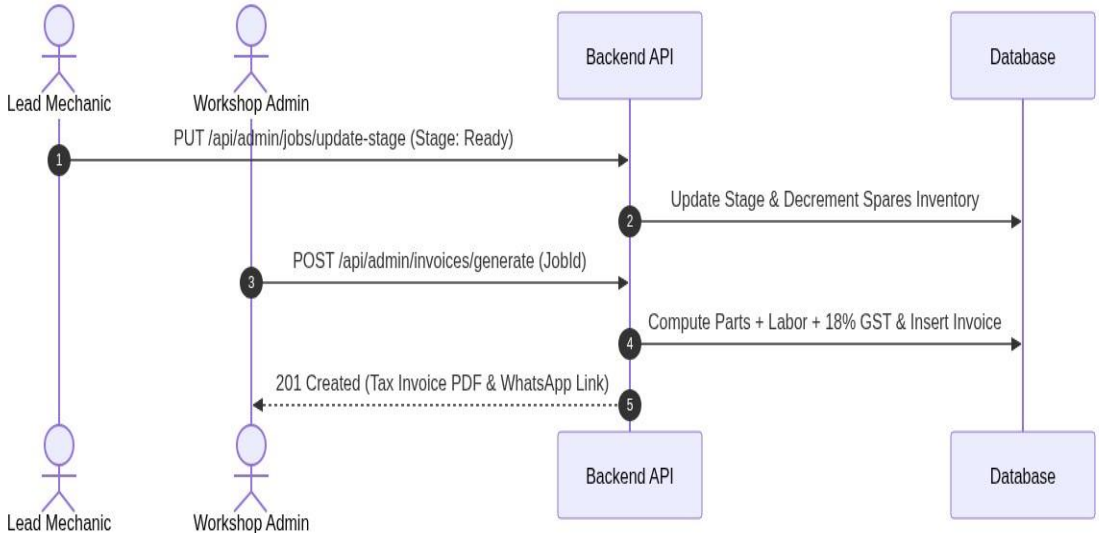


Figure 3.3.2 – Sequence Diagram (Job Card Creation & Inventory Allocation)

3.4 Entity-Relationship (ER) Diagram (Enterprise Domain Entities)

The relational database schema is modeled in Third Normal Form (3NF). Figure 3.4.1 illustrates the enterprise domain entities (Users, Vehicles, JobCards, InventoryItems, Invoices, Appointments), enforcing referential integrity via foreign key constraints.

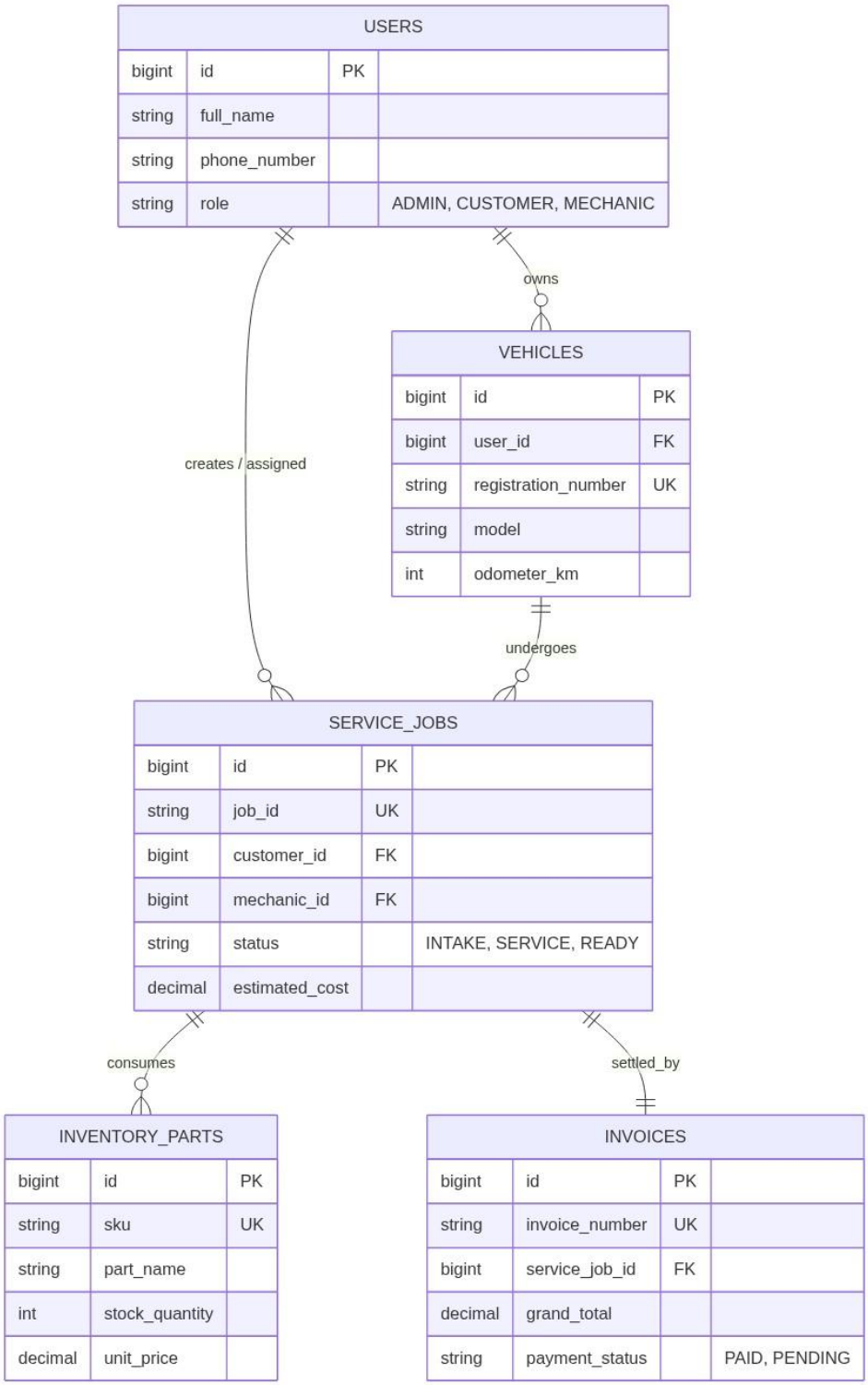


Figure 3.4.1 – (ER) Diagram (Enterprise Domain Entities)

3.4 Entity-Relationship (ER) Diagram (Workshop Operations & Spares Ledger)

Figure 3.4.2 highlights the operational core linking Job Cards, Consumed Parts (job_card_parts), and the central Inventory Store. Each line item captures unit price and quantity snapshots to ensure historical billing accuracy even if future catalog prices change.

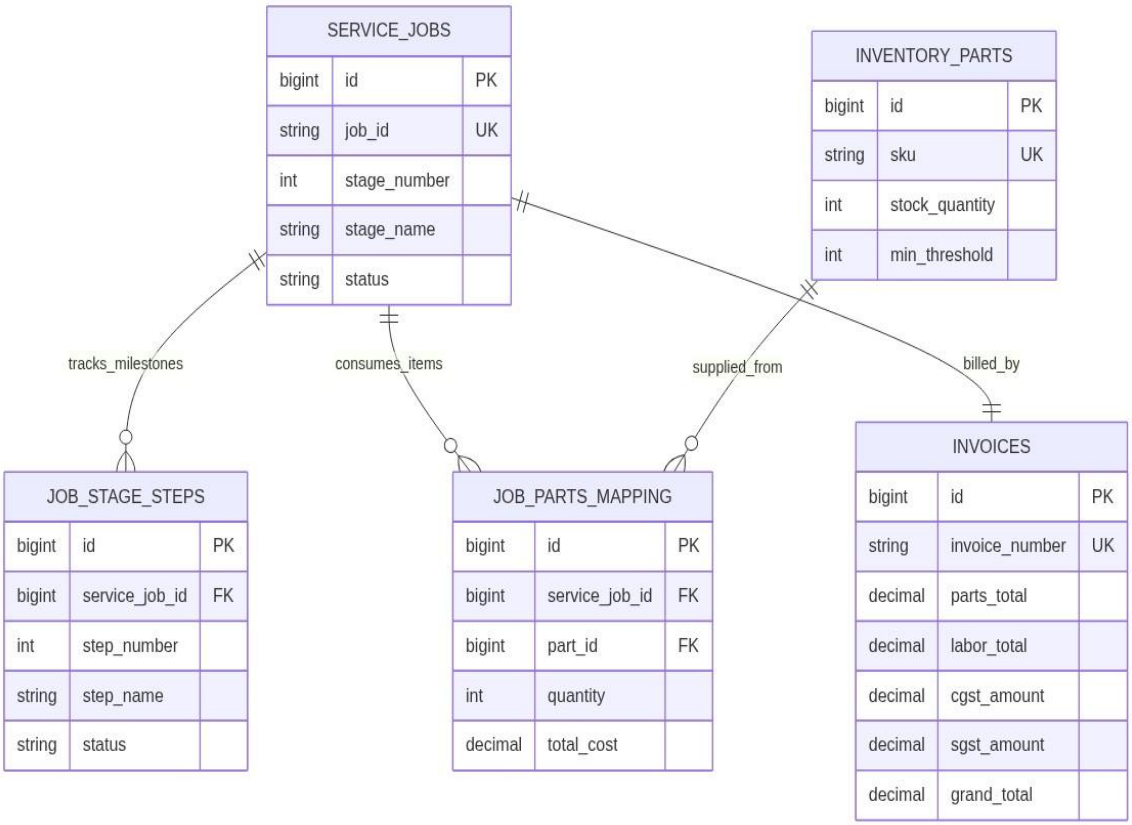


Figure 3.4.2 – (ER) Diagram (Workshop Operations & Spares Ledger)

3.5 Deployment Diagram (Cloud Multi-Tier Topology)

Figure 3.5 illustrates the production multi-cloud deployment topology. Client browsers interact with the React Single Page Application delivered via Vercel Edge CDN nodes. API calls are securely routed to the Spring Boot application container on Render, which connects to the managed Supabase PostgreSQL database in the AWS Mumbai region over encrypted TLS.

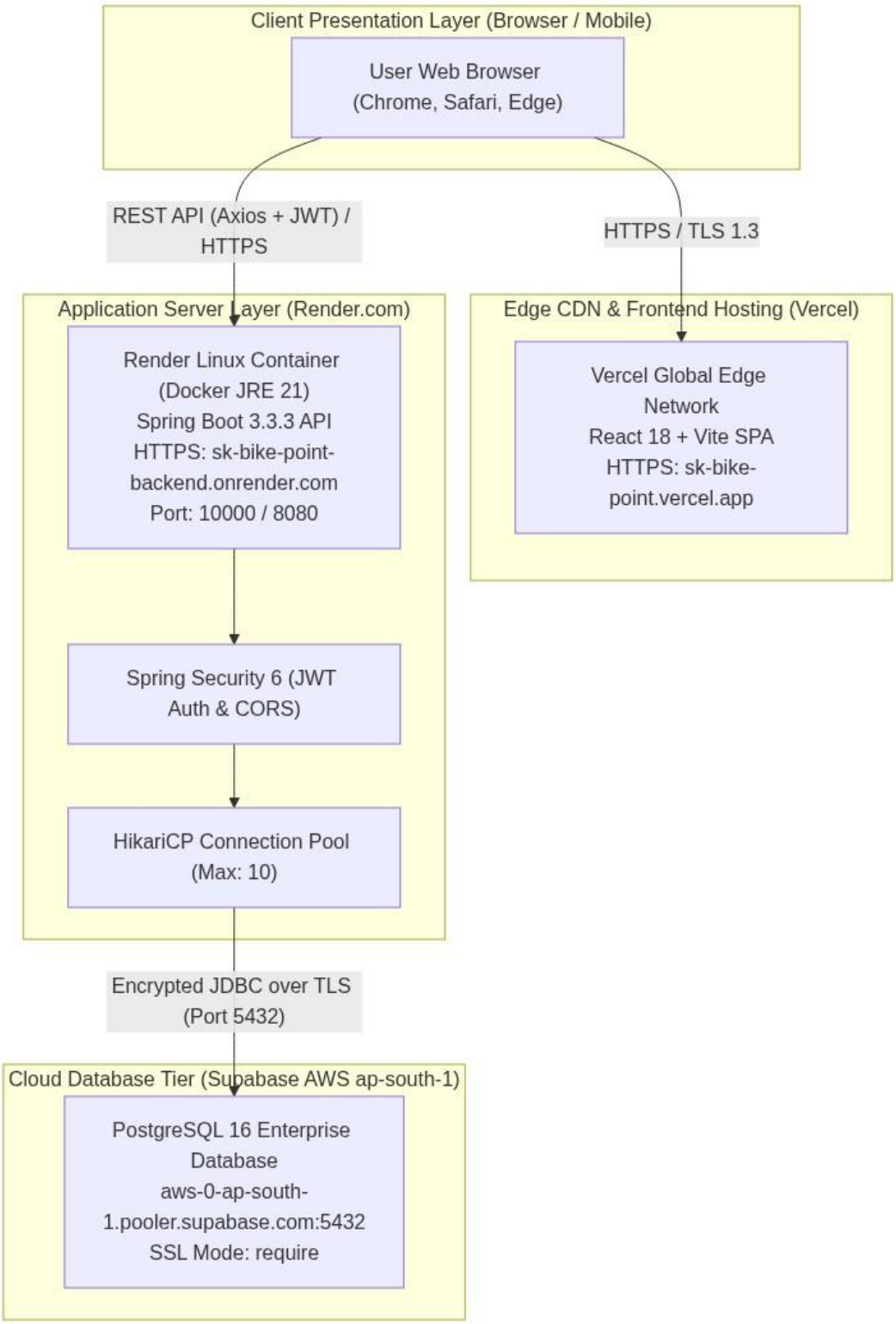


Figure 3.5 – Deployment Diagram (Cloud Multi-Tier Topology)

3.6 Security & Data Flow Architecture

The S K BIKE POINT system incorporates multi-layered defense-in-depth principles across transport, authentication, authorization, and data storage layers to safeguard customer information, repair logs, and business financial metrics.

A. Stateless JWT Authentication & RBAC Flow:

Upon successful user credential verification against BCrypt password hashes, the authentication service generates a cryptographically signed JSON Web Token (JWT) using the HMAC-SHA256 algorithm. The token encapsulates claims including user email, issued timestamp, expiration time (24-hour TTL), and user roles (ROLE_ADMIN or ROLE_CUSTOMER). Each subsequent client HTTP request transmits this token in the Authorization Bearer header. The Spring Security JwtAuthenticationFilter intercepts and validates the token signature before delegating to the endpoint.

B. Database Connection Pooling & Transport Encryption:

All client-to-server and server-to-database communication channels are strictly encrypted using TLS 1.3 / HTTPS. The backend leverages the HikariCP connection pool with a maximum pool size of 10 connections, maintaining an optimal pool threshold with aggressive connection leak detection (leak-detection-threshold = 20,000 ms) and connection timeouts (30,000 ms) to resist denial-of-service attempts.

Table 3.6.1: Enterprise Security Dimension Matrix

Security Dimension	Implementation Mechanism	Security Objective
Credential Protection	BCrypt Hashing (Cost factor 10 with salt)	Prevents credential extraction even in database dumps
Session Management	Stateless JWT Bearer Token (HMAC-SHA256)	Eliminates server session state; resists CSRF attacks
Network Transport	Enforced HTTPS / TLS 1.3 Encryption	Prevents eavesdropping and man-in-the-middle sniffing
CORS Policy	Restricted Allowed Origins (Vercel & Localhost)	Blocks unauthorized cross-domain API invocation
Input Sanitization	Jakarta Validation (@NotBlank, @Email, regex)	Prevents SQL injection, XSS, and buffer overflows

Chapter 4
SYSTEM DESCRIPTION

4.1 Database Description Tables

Table 4.1.1: USERS

Field	Data Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
full_name	VARCHAR(100)	NOT NULL	User's full legal name
email	VARCHAR(100)	UNIQUE, NOT NULL	Login email address
password	VARCHAR(255)	NOT NULL	BCrypt hashed password string
phone	VARCHAR(20)	NULLABLE	Primary contact mobile number
role	VARCHAR(20)	NOT NULL	ROLE_ADMIN or ROLE_CUSTOMER
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Account creation timestamp
updated_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Last profile modification timestamp

Table 4.1.2: VEHICLES

Field	Data Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique vehicle record identifier
user_id	BIGINT	FOREIGN KEY (users.id)	Registered vehicle owner reference
registration_number	VARCHAR(20)	UNIQUE, NOT NULL	License plate number (e.g. MH-12-AB-1234)
brand	VARCHAR(50)	NOT NULL	Two-wheeler make (Royal Enfield, Honda, etc.)
model	VARCHAR(50)	NOT NULL	Vehicle model (Classic 350, Activa 6G, etc.)
manufacturing_year	INTEGER	NULLABLE	Year of vehicle manufacture
color	VARCHAR(30)	NULLABLE	Vehicle exterior color
fuel_type	VARCHAR(20)	NULLABLE	Fuel classification (Petrol, EV, Hybrid)

mileage_km	INTEGER	NULLABLE	Current odometer reading in kilometers

Table 4.1.3: INVENTORY_ITEMS

Field	Data Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique spare item identifier
part_number	VARCHAR(50)	UNIQUE, NOT NULL	Unique SKU code (e.g. MOT-5100-15W50)
name	VARCHAR(100)	NOT NULL	Commercial part or lubricant description
category	VARCHAR(50)	NOT NULL	Category (Engine Oils, Brakes, Filters)
stock_quantity	INTEGER	NOT NULL, DEFAULT 0	Current available shelf stock count
min_threshold	INTEGER	NOT NULL, DEFAULT 5	Minimum replenishment trigger level
unit_price	NUMERIC(10,2)	NOT NULL	Selling retail price per unit in INR
location_bin	VARCHAR(50)	NULLABLE	Physical shelf or bin location (e.g. Rack A-1)

Table 4.1.4: JOB_CARDS

Field	Data Type	Constraint	Description
id	BIGINT	PRIMARY KEY, AUTO_INCREMENT	Unique job card identifier
job_number	VARCHAR(30)	UNIQUE, NOT NULL	Formatted Job Code (e.g. JOB-2026-081)
customer_id	BIGINT	FOREIGN KEY (users.id)	Customer associated with service
vehicle_id	BIGINT	FOREIGN KEY (vehicles.id)	Vehicle undergoing repair
assigned_mechanic	VARCHAR(100)	NULLABLE	Master technician assigned to job
status	VARCHAR(30)	NOT NULL	IN_PROGRESS, READY, DELIVERED
stage_number	INTEGER	NOT NULL, DEFAULT 1	Active repair stage index (1 to 5)
stage_name	VARCHAR(100)	NOT NULL	Title of active repair stage
diagnostic_notes	TEXT	NULLABLE	Technical diagnostic checklist notes

labor_charges	NUMERIC(10,2)	NOT NULL, DEFAULT 0	Service labor charge amount in INR
parts_total	NUMERIC(10,2)	NOT NULL, DEFAULT 0	Total price of consumed spare parts
grand_total	NUMERIC(10,2)	NOT NULL, DEFAULT 0	Total job card billing amount

Table 4.1.5: JOB_CARD_PARTS & Table 4.1.6: INVOICES & APPOINTMENTS

Table	Key Fields	Data Types	Constraints & Purpose
job_card_parts	id, job_card_id, inventory_item_id, part_name, quantity, unit_price, total_price	BIGINT, BIGINT, BIGINT, VARCHAR, INT, NUMERIC, NUMERIC	Links allocated inventory items to specific job cards with quantity and price snapshot.
invoices	id, invoice_number, job_card_id, customer_id, subtotal, tax_amount, grand_total, payment_status	BIGINT, VARCHAR, BIGINT, BIGINT, NUMERIC, NUMERIC, NUMERIC, VARCHAR	Tracks GST 18% tax billing, payment modes (Cash, UPI, Card), and settlement records.
appointments	id, customer_name, phone, bike_model, service_type, appointment_date, time_slot, status	BIGINT, VARCHAR, VARCHAR, VARCHAR, VARCHAR, DATE, VARCHAR, VARCHAR	Manages rider service appointments booked online from the public landing page.

4.2 Module Description

1. Zero-Login Live Vehicle Tracker Module: Enables any rider to input their license plate or job code on the public homepage to immediately stream 5-stage mechanical repair status, assigned technician, and parts breakdown without login barriers.

2. Workshop Command Center & Analytics Module: Displays 6 core workshop operational KPIs (Total Customers, Active Jobs, Registered Bikes, Daily Revenue, Low-Stock Parts, Pending Bills) with interactive Recharts volume trends.

3. Digital Job Card Management Module: Enables service advisors to initiate jobs, assign technicians, link customer bikes, record diagnostic checklists, and advance live repair stages.

4. Spare Parts & Inventory Control Module: Manages real-time inventory balances, SKU categories, minimum reorder triggers, physical bin indexing, and automatic decrementing upon job allocation.

5. GST 18% Invoicing & Settlement Module: Automates tax computation (9% CGST + 9% SGST), labor breakdown, promotional discount application, and professional printable tax bills.

6. Customer Self-Service Portal & Garage: Secure customer area to view owned vehicles, mileage records, service history archives, and preventive maintenance reminders.

7. Role-Based Authentication & Security Module: Stateless Spring Security with BCrypt password hashing and JWT token issuance enforcing ROLE_ADMIN and ROLE_CUSTOMER boundaries.

8. Service Appointment Booking Module: Public booking widget allowing customers to reserve service bays by selecting preferred dates, time slots, and two-wheeler models.

9. Automated Service History Ledger Module: Maintains indelible maintenance records for every vehicle to verify resale value, past part replacements, and periodic intervals.

10. REST API Gateway & Global Exception Module: Centralized Spring Boot exception handler delivering standard ApiResponse envelopes and error status codes.

4.3 System Runtime Output

The web platform delivers a responsive, modern user experience across both public and role-protected workshop views. Figure 4.1 shows the high-conversion public landing page featuring the zero-login vehicle tracking widget, 1-click booking, and customer testimonials.

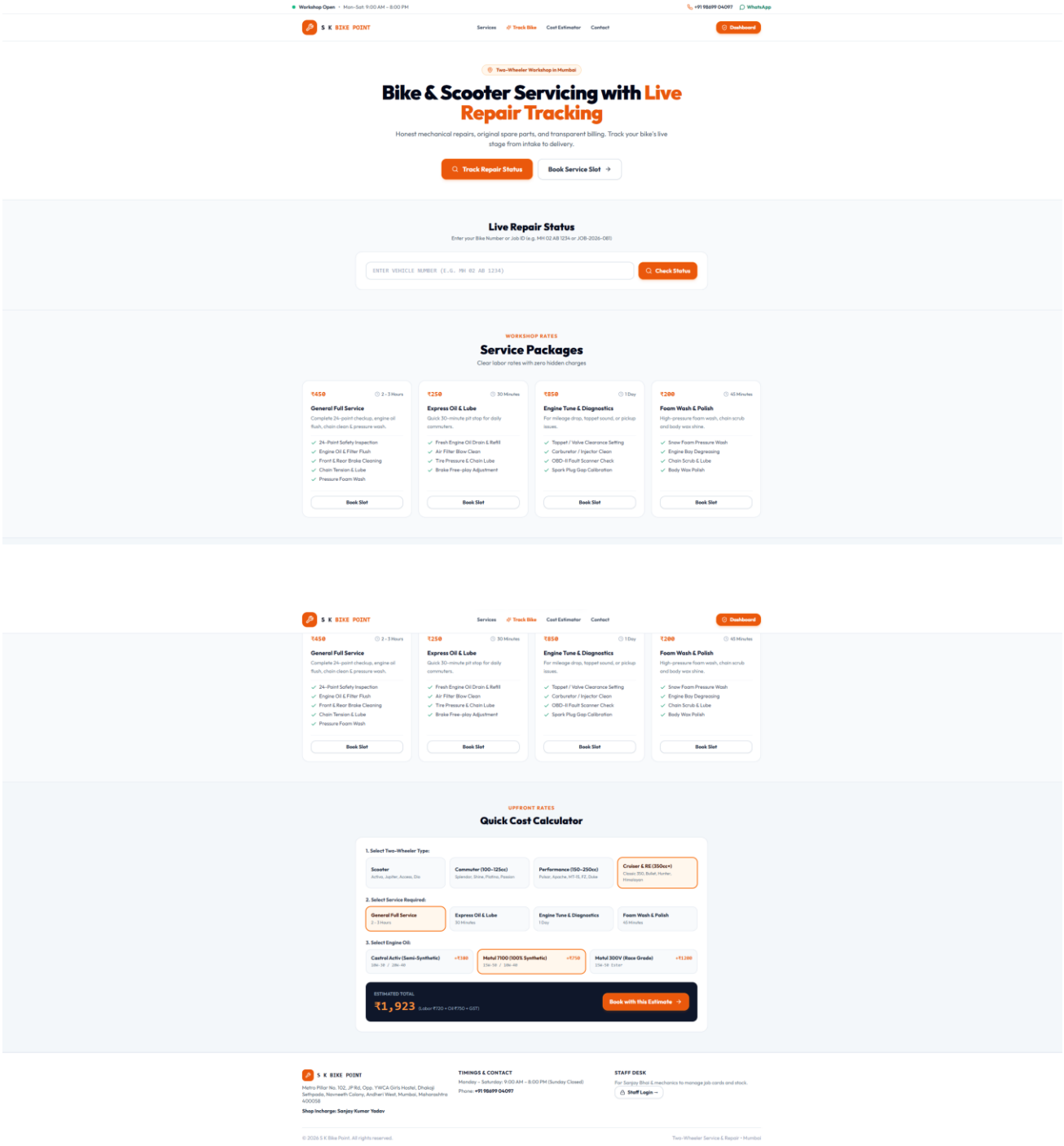


Figure 4.1 – Home Page & Zero-Login Live Vehicle Tracker

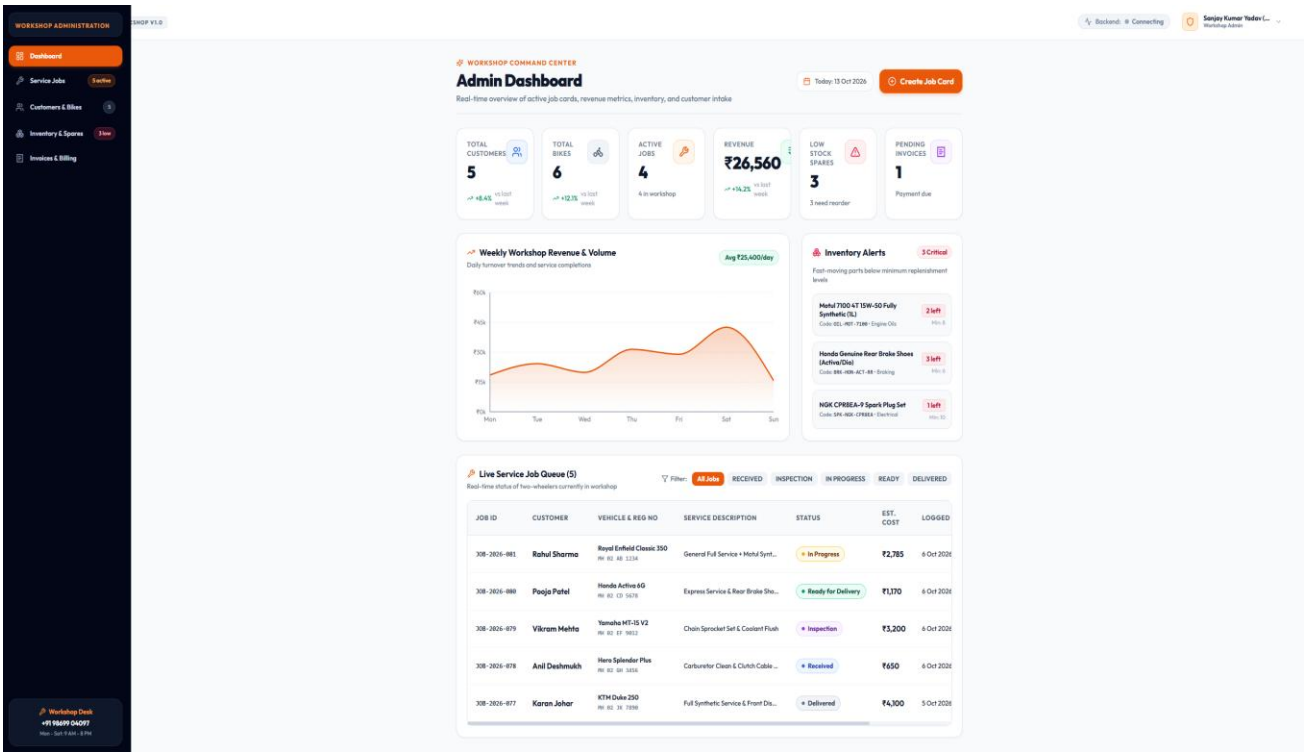


Figure 4.2 – Workshop Administrator Command Center & Live Analytics

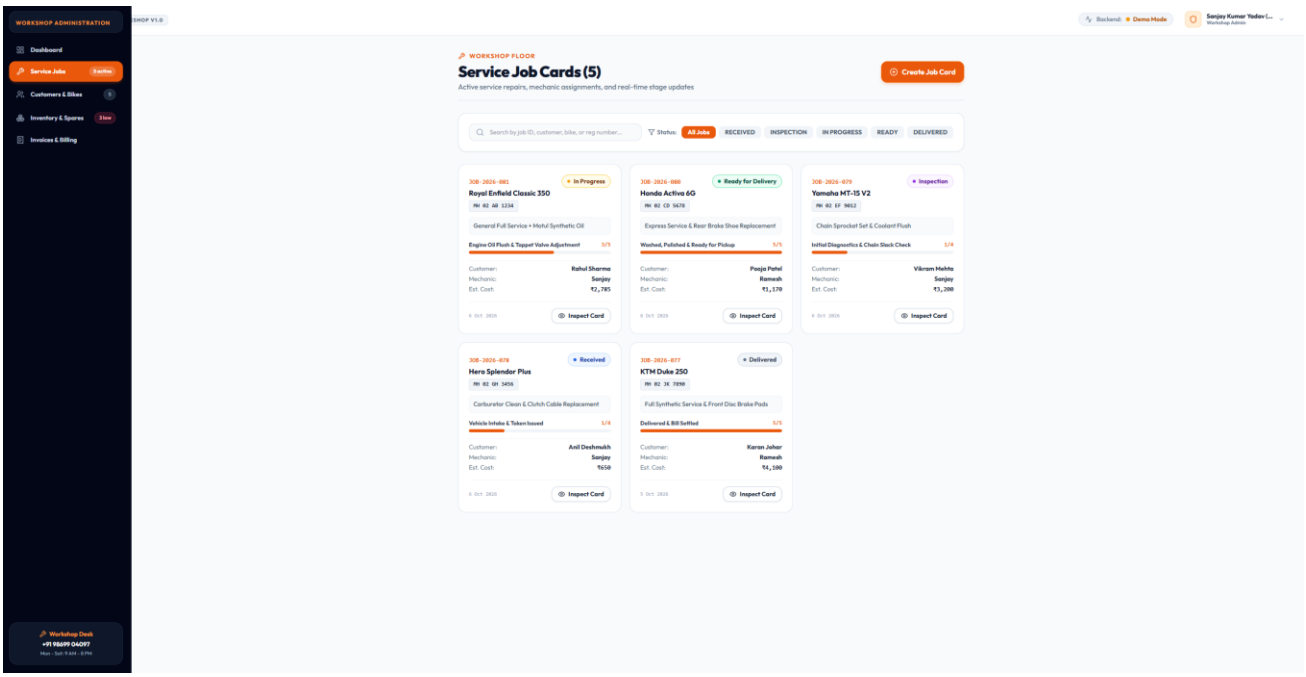


Figure 4.3 – Service Job Queue & Active Workshop Floor Cards

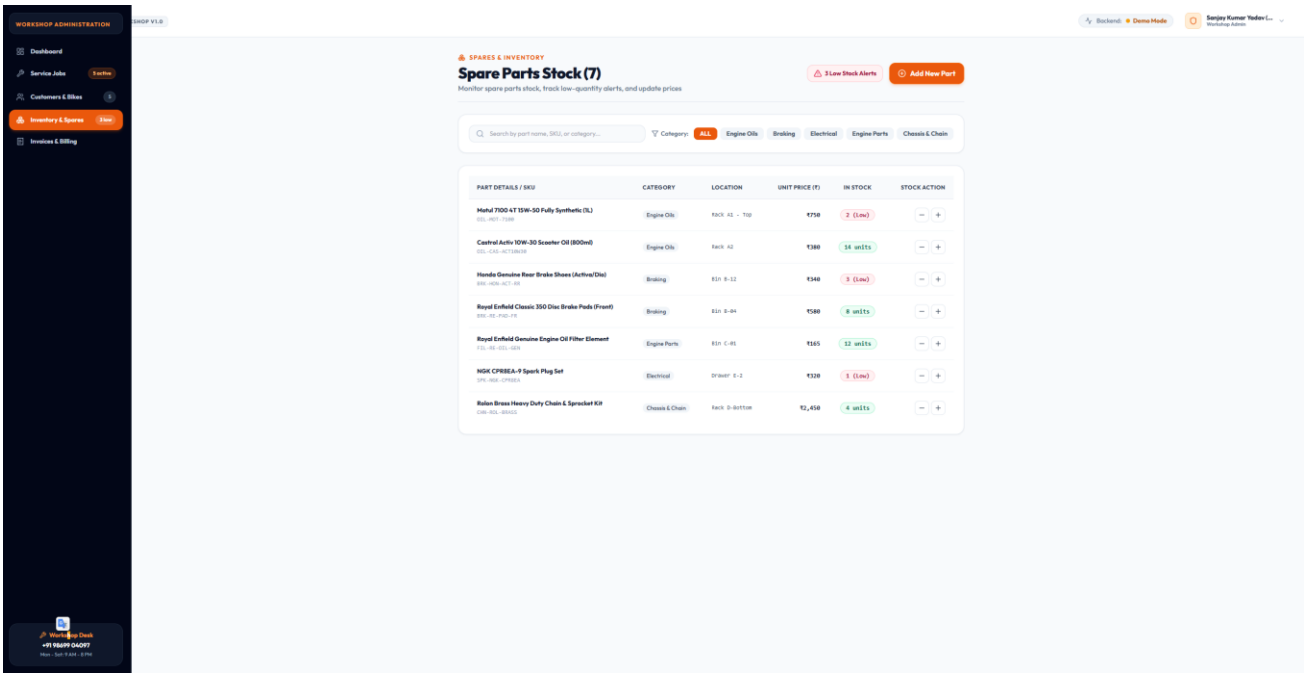


Figure 4.4 – Spare Parts Inventory Ledger & Critical Low-Stock Alerts

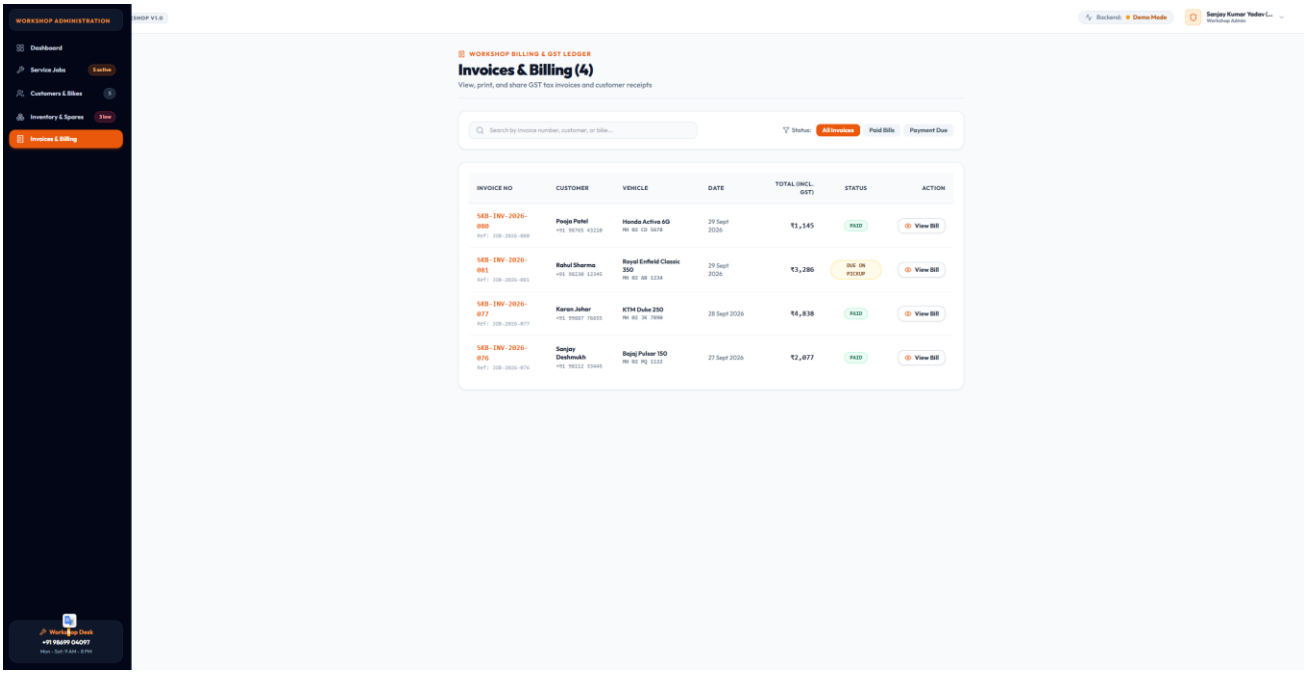


Figure 4.5 – GST 18% Tax Invoicing & Settlement Management

4.4 Coding (Key Source Snippets)

1. Database Connection & HikariCP Configuration (backend/src/main/resources/application.yml)

```
spring:
  datasource:
    url: ${DATABASE_URL:jdbc:postgresql://aws-0-ap-south-
1.pooler.supabase.com:5432/postgres?sslmode=require}
    username: ${DATABASE_USERNAME:postgres.rfuhtriyynsiccgvnxsk}
    password: ${DATABASE_PASSWORD:sum3et@skbikepoint}
    driver-class-name: org.postgresql.Driver
    hikari:
      maximum-pool-size: 10
      minimum-idle: 3
      connection-timeout: 30000
      idle-timeout: 600000
      max-lifetime: 1800000
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: false
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect
        format_sql: true
```

2. User Authentication & JWT Generation (backend/src/main/java/com/skbikepoint/service/AuthService.java)

```
@Service
@RequiredArgsConstructor
public class AuthService {
    private final AuthenticationManager authenticationManager;
    private final JwtTokenProvider tokenProvider;
    private final UserRepository userRepository;

    public AuthResponse login(LoginRequest request) {
        Authentication auth = authenticationManager.authenticate(
            new
UsernamePasswordAuthenticationToken(request.getEmail(),
request.getPassword())
        );

SecurityContextHolder.getContext().setAuthentication(auth);
        String token = tokenProvider.generateToken(auth);
        User user =
userRepository.findByEmail(request.getEmail())
```

```
            .orElseThrow(() -> new
ResourceNotFoundException("User not found"));
        return new AuthResponse(token, "Bearer",
UserDto.fromEntity(user));
    }
}
```

3. JWT Token Provider Implementation
(backend/src/main/java/com/skbikepoint/security/JwtTokenProvider.java)

```
@Component
public class JwtTokenProvider {
    @Value("${jwt.secret}")
    private String jwtSecret;

    @Value("${jwt.expiration-ms}")
    private long jwtExpirationMs;

    public String generateToken(Authentication authentication) {
        UserPrincipal userPrincipal = (UserPrincipal)
authentication.getPrincipal();
        Date now = new Date();
        Date expiryDate = new Date(now.getTime() +
jwtExpirationMs);
        SecretKey key =
Keys.hmacShaKeyFor(jwtSecret.getBytes(StandardCharsets.UTF_8));

        return Jwts.builder()
                .subject(userPrincipal.getEmail())
                .claim("role", userPrincipal.getRole())
                .issuedAt(now)
                .expiration(expiryDate)
                .signWith(key)
                .compact();
    }
}
```

4. Zero-Login Public Vehicle Tracker
(backend/src/main/java/com/skbikepoint/controller/JobCardController.java)

```
@RestController
@RequestMapping("/api")
@RequiredArgsConstructor
public class JobCardController {
    private final JobCardService jobCardService;

    @GetMapping("/public/jobs/track/{query}")
    public ResponseEntity<ApiResponse<JobCardDto>>
trackJobPublic(@PathVariable String query) {
        JobCardDto job = jobCardService.trackJob(query);
```

```
        return ResponseEntity.ok(ApiResponse.success(job, "Job
tracking retrieved successfully"));
    }
}
```

5. Job Card Service Query Logic
(backend/src/main/java/com/skbikepoint/service/JobCardServiceImpl.java)

```
@Override
@Transactional(readOnly = true)
public JobCardDto trackJob(String query) {
    String cleanQuery = query.replaceAll("[\\s-]",
"").toUpperCase();
    return
jobCardRepository.findActiveJobByPlateOrJobNumber(cleanQuery)
        .map(JobCardDto::fromEntity)
        .orElseThrow(() -> new ResourceNotFoundException(
            "No active service job found for registration or job
number: " + query
        ));
}
```

6. Inventory Allocation & Stock Decrement
(backend/src/main/java/com/skbikepoint/service/JobCardServiceImpl.java)

```
@Override
@Transactional
public JobCardDto addPartToJob(Long jobId, AddPartRequest
request) {
    JobCard jobCard = jobCardRepository.findById(jobId)
        .orElseThrow(() -> new ResourceNotFoundException("Job
card not found"));

    InventoryItem item =
inventoryRepository.findById(request.getInventoryItemId())
        .orElseThrow(() -> new ResourceNotFoundException("Part
not found"));

    if (item.getStockQuantity() < request.getQuantity()) {
        throw new InsufficientStockException("Insufficient stock
for " + item.getName());
    }

    // Decrement inventory stock
    item.setStockQuantity(item.getStockQuantity() -
request.getQuantity());
    inventoryRepository.save(item);

    // Append part line item to job card
    JobCardPart part = JobCardPart.builder()
        .jobCard(jobCard)
```

```
        .inventoryItem(item)
        .partName(item.getName())
        .quantity(request.getQuantity())
        .unitPrice(item.getUnitPrice())

.totalPrice(item.getUnitPrice().multiply(BigDecimal.valueOf(reque
st.getQuantity())))
        .build();
    jobCardPartRepository.save(part);
    return JobCardDto.fromEntity(jobCard);
}
```

7. Automatic Low-Stock Detection
(backend/src/main/java/com/skbikepoint/service/InventoryService.java)

```
@Override
@Transactional(readOnly = true)
public List<InventoryItemDto> getLowStockAlerts() {
    return inventoryRepository.findLowStockItems()
        .stream()
        .map(InventoryItemDto::fromEntity)
        .collect(Collectors.toList());
}
```

8. GST 18% Tax Invoicing Engine
(backend/src/main/java/com/skbikepoint/service/InvoiceServiceImpl.java)

```
@Override
@Transactional
public InvoiceDto generateInvoiceForJob(Long jobId, String
paymentMethod) {
    JobCard job = jobCardRepository.findById(jobId)
        .orElseThrow(() -> new ResourceNotFoundException("Job not
found"));

    BigDecimal labor = job.getLaborCharges() != null ?
job.getLaborCharges() : BigDecimal.ZERO;
    BigDecimal parts = job.getPartsTotal() != null ?
job.getPartsTotal() : BigDecimal.ZERO;
    BigDecimal subtotal = labor.add(parts);

    // GST Calculation: 18% Total (9% CGST + 9% SGST)
    BigDecimal gstRate = new BigDecimal("0.18");
    BigDecimal taxAmount = subtotal.multiply(gstRate).setScale(2,
RoundingMode.HALF_UP);
    BigDecimal grandTotal = subtotal.add(taxAmount);

    Invoice invoice = Invoice.builder()
        .invoiceNumber("INV-2026-" + String.format("%04d",
job.getId()))
```

```
        .jobCard(job)
        .customer(job.getCustomer())
        .subtotal(subtotal)
        .taxAmount(taxAmount)
        .grandTotal(grandTotal)
        .paymentStatus("PAID")
        .paymentMethod(paymentMethod)
        .paymentDate(LocalDateTime.now())
        .build();

    return
InvoiceDto.fromEntity(invoiceRepository.save(invoice));
}
```

9. Invoice Entity Data Model (backend/src/main/java/com/skbikepoint/entity/Invoice.java)

```
@Entity
@Table(name = "invoices")
@Getter @Setter @NoArgsConstructor @AllArgsConstructor @Builder
public class Invoice extends BaseEntity {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true, length = 30)
    private String invoiceNumber;

    @OneToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "job_card_id", nullable = false, unique =
true)
    private JobCard jobCard;

    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "customer_id", nullable = false)
    private User customer;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal subtotal;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal taxAmount;

    @Column(nullable = false, precision = 10, scale = 2)
    private BigDecimal grandTotal;
}
```

10. Spring Security 6 Filter Middleware (backend/src/main/java/com/skbikepoint/security/SecurityConfig.java)

```
@Configuration
@EnableWebSecurity
```

```
@RequiredArgsConstructor
public class SecurityConfig {
    private final JwtAuthenticationFilter jwtFilter;

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http)
throws Exception {
        return http
            .cors(Customizer.withDefaults())
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s ->
s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/api/health", "/api/auth/**",
"/api/public/**").permitAll()
                .requestMatchers("/api/admin/**",
"/api/inventory/**").hasRole("ADMIN")

.requestMatchers("/api/customer/**").hasRole("CUSTOMER")
                .anyRequest().authenticated()
            )
            .addFilterBefore(jwtFilter,
UsernamePasswordAuthenticationFilter.class)
            .build();
    }
}
```

Chapter 5
TESTING

5.1 Testing Methodology and Environment

The S K BIKE POINT workshop system was rigorously verified using black-box functional testing, white-box integration testing, and automated security penetration checks. Test environments encompassed local developer containers and the live cloud deployment infrastructure.

Item	Details
Operating Systems	Windows 11 Professional & Ubuntu Linux 22.04 LTS
Web Browsers	Google Chrome 128+, Microsoft Edge 128+, Mozilla Firefox 130+
Frontend Engine	React 18.3, Vite 5.4, TypeScript 5.5, Tailwind CSS 3.4
Backend Framework	Java 21 LTS, Spring Boot 3.3.3, Spring Security 6
Database Tier	Managed PostgreSQL 16 on Supabase (AWS Mumbai ap-south-1)
Connection Pooler	HikariCP (Max 10 pool connections over TLS 1.3)
Cloud Deployment	Vercel Global Edge (Frontend) & Render Cloud Container (Backend)
Testing Devices	Desktop Counter (1920x1080), Tablet (768x1024), Mobile Viewports
Security Testing	JWT expiry validation, role authorization guards, SQL injection guards

5.2 Unit, Integration and Security Testing

1. Unit Testing: Verified isolated service calculations including 18% GST tax computation, stock decrement mathematics, JWT signing and expiry validation, and DTO conversion logic.

2. Integration Testing: Verified end-to-end flows connecting the React frontend to Spring Boot REST endpoints and the Supabase PostgreSQL database, ensuring database transactions roll back on failure.

3. Security and Authorization Testing: Tested unauthenticated access attempts against protected /api/admin/** and /api/customer/** endpoints to confirm 401 Unauthorized and 403 Forbidden enforcement.

4. Robustness & Exception Handling Testing: Simulated network drops, connection pool timeouts, and invalid SQL injections to confirm structured error response envelopes and 500 error shielding.

5.3 Master Test Case Matrix

ID	Feature	Input / Action	Expected Result	Actual Result	Result
TC-01	Zero-Login Tracker	Plate: MH-12-AB-1234	Returns active job card with 5-stage progress stepper	Live stepper rendered with diagnostic notes	PASS
TC-02	Unknown Plate Query	Plate: DL-01-XX-9999	Returns 404 NOT_FOUND with helpful contact banner	Displayed workshop help desk call banner	PASS
TC-03	Admin Login	Valid email & BCrypt pwd	HTTP 200 with JWT Bearer token & Admin role claims	Token saved; navigated to Command Center	PASS
TC-04	Invalid Login	Incorrect password	HTTP 401 with 'Bad credentials' error response	Displayed 'Invalid credentials' notification toast	PASS
TC-05	Create Job Card	Plate, Bike Model, Mechanic	New job generated with status IN_PROGRESS & stage 1	Job persisted with unique Job Number code	PASS
TC-06	Inventory Deduction	Add 2 spark plugs to job	Job total updated; shelf stock decremented by 2	Inventory updated; line item saved in job_parts	PASS
TC-07	Insufficient Stock	Requisition 100 units (qty > 8)	Throws InsufficientStockException (HTTP 400)	Displayed 'Insufficient inventory stock' alert	PASS
TC-08	Update Repair Stage	Advance to 'Stage 3: Assembly'	Stage persisted; live public tracker reflects change	Tracker stepper turns green on Stage 3	PASS
TC-09	18% GST Invoice	Complete job & click Settle	Subtotal calculated; 9% CGST + 9% SGST added	Grand total accurate; Invoice marked PAID	PASS
TC-10	Role Guard Check	Customer calls /api/admin/**	HTTP 403 Forbidden returned by Spring Security	Access blocked; redirected to customer portal	PASS

5.3 Master Test Case Matrix (Continued)

ID	Feature	Input / Action	Expected Result	Actual Result	Result
TC-11	Customer Garage	View owned vehicles list	Displays bikes registered to customer ID with history	Rendered owned bikes and past completed jobs	PASS
TC-12	Book Appointment	Submit date, slot, bike model	Creates appointment record with status PENDING	Appointment saved; confirmed on rider screen	PASS
TC-13	Thermal Slip View	Print 80mm invoice receipt	Renders compact printable gate pass layout	Formatted 80mm slip layout displayed in modal	PASS
TC-14	Tampered JWT	Alter signature in Bearer token	HTTP 401 Unauthorized; token rejection	Request blocked; session cleared immediately	PASS
TC-15	Concurrent Stock	Simultaneous requests for last unit	One succeeds, second receives stock exception	Database row lock prevented negative inventory	PASS

Chapter 6
CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

The S K BIKE POINT two-wheeler workshop and customer management platform provides a modern, cloud-native solution for automobile repair centers. By replacing manual paperwork with automated digital job cards, providing zero-login live vehicle tracking, linking real-time inventory deductions directly to repairs, and automating 18% GST compliance invoicing, the system eliminates administrative bottlenecks, prevents spare parts leakage, and delivers full transparency to vehicle owners.

Developed with a responsive React 18 and TypeScript frontend, a secure Spring Boot 3 REST API backend, and cloud persistence on Supabase PostgreSQL with HikariCP connection pooling, the platform offers reliability, enterprise security, and high performance across mobile, tablet, and desktop counter environments.

6.2 Future Scope

- **Automated WhatsApp Notifications:** Integrate the WhatsApp Business Cloud API to automatically dispatch repair stage alerts, diagnostic photos, and downloadable PDF bills directly to customer phones.
- **80mm Thermal Slip Printing:** Provide direct USB/Bluetooth thermal printer integration for printing compact 80mm gate passes and counter payment receipts.
- **Mechanic Commission & Performance Tracking:** Implement automated technician commission tracking based on billable labor charges to incentivize workshop efficiency.
- **OBD-II Bluetooth Scanner Interface:** Integrate Bluetooth OBD-II motorcycle diagnostic scanners to automatically ingest engine fault codes (DTCs) into digital job cards.
- **Multi-Branch Franchise Architecture:** Extend the database model to support multi-branch workshop chains with centralized inventory replenishment and franchise analytics.
- **Native Mobile Applications:** Develop dedicated Android and iOS native mobile apps using React Native for mechanics and fleet managers.

Chapter 7
UNDERTAKING

I, Sumeet Yadav, a candidate for the degree of Bachelor of Science in Computer Science, hereby undertake that:

1. The project work presented in "S K BIKE POINT: Two-Wheeler Workshop & Customer Management System" is my original work, carried out under academic supervision.

2. The source code, database structure, system design, UML diagrams, and related documentation were developed specifically for this project.

3. Proper acknowledgement has been given for all third-party libraries, frameworks, tools, and technologies used in the project, including Java 21, Spring Boot 3, React 18, TypeScript, Tailwind CSS, PostgreSQL 16, Supabase, JWT, BCrypt, Docker, Render, and Vercel, along with all references and resources used.

Date: _____ / ___ / _____

Place: Mumbai

_____

Signature Of the Student

Sumeet Yadav

Chapter 8
BIBLIOGRAPHY AND REFERENCES

- **Spring Boot Documentation:** Building Production-Grade Spring Boot 3 Applications & REST APIs, VMware Tanzu.
  https://spring.io/projects/spring-boot
- **React Framework Documentation:** React 18 Single Page Applications & Functional Components, Meta Open Source.
  https://react.dev
- **TypeScript Language Guide:** Typed JavaScript at Any Scale, Microsoft Corporation.
  https://www.typescriptlang.org/docs/
- **PostgreSQL Global Development Group:** PostgreSQL 16 Relational Database Management System Documentation.
  https://www.postgresql.org/docs/
- **Supabase Documentation:** Managed Cloud PostgreSQL, Connection Pooling & Enterprise Security, Supabase Inc.
  https://supabase.com/docs
- **Tailwind CSS Documentation:** Utility-First CSS Framework, Tailwind Labs.
  https://tailwindcss.com/docs
- **Auth0 & JJWT Documentation:** JSON Web Tokens (JWT) Architecture & HMAC-SHA256 Implementation.
  https://jwt.io/
- **Vercel Documentation:** Deploying Modern Single Page Applications on Edge CDN Infrastructure.
  https://vercel.com/docs
- **Render Cloud Platform:** Containerized Web Services & Automated CI/CD Deployments.
  https://render.com/docs
- **Docker Documentation:** Containerization, Multi-Stage Builds & Docker Compose Specification.
  https://docs.docker.com/
- **GitHub Documentation:** Git Version Control & Repository Management.
  https://docs.github.com/